kauno kolegija | Technologijų fakultetas

Swagger / OpenAPI .NET API

SimpleStudentApi dokumentavimas, testavimas ir naudojimas

Grupė PS-5 | 2026 m. ruduo | Paskaita + praktika

Dėstytojas: Tadas Bulis

TURINYS

Ką šiandien aptarsime

Penki blokai – paskutinis yra jūsų praktika

1. **Kas yra Swagger ir OpenAPI**
   Sąvokos ir kodėl API reikia kontrakto

2. **Swagger įdiegimas API**
   Vienas paketas, keturios konfigūracijos eilutės, API metodų aprašymas

3. **Darbas su Swagger UI**
   Skaityti, išbandyti, matyti klaidas, kopijuoti curl

4. **Kontrakto naudojimas frontend dalyje**
   Sugeneruotas C# klientas ir PHP klientas, vykdymo metu skaitantis tą patį kontraktą

5. **Praktika ir jūsų dokumentacija**
   Laboratorinės užduotys ir OpenAPI vieta projekto dokumentacijoje

OpenAPI – tai kontraktas tarp serverio dalies (backend) ir visų jos klientų: vienas mašinai suprantamas failas, kurį skaito žmonės, testuoja įrankiai ir iš kurio generatoriai sukuria kliento kodą.

EN: OpenAPI – API contract · Swagger UI – its interactive viewer

SĄVOKOS

Keturi pavadinimai, kuriuos išgirsite

Jie dažnai painiojami – štai kas yra kas

1 OpenAPI specifikacija

Standartas (OAS 3.x) HTTP API aprašyti JSON arba YAML formatu: keliai, parametrai, užklausų turinys, atsakymai, schemos. Nepriklausomas nuo gamintojo, prižiūrimas Linux Foundation.

2 Swagger

Pradinis specifikacijos pavadinimas (iki 2015 m.), dabar – SmartBear įrankių šeima. Swagger UI – tinklalapis, atvaizduojantis OpenAPI failą.

3 Swashbuckle.AspNetCore

Mūsų API naudojamas NuGet paketas: nuskaito .NET API metodus, sugeneruoja swagger.json ir pateikia Swagger UI adresu /swagger.

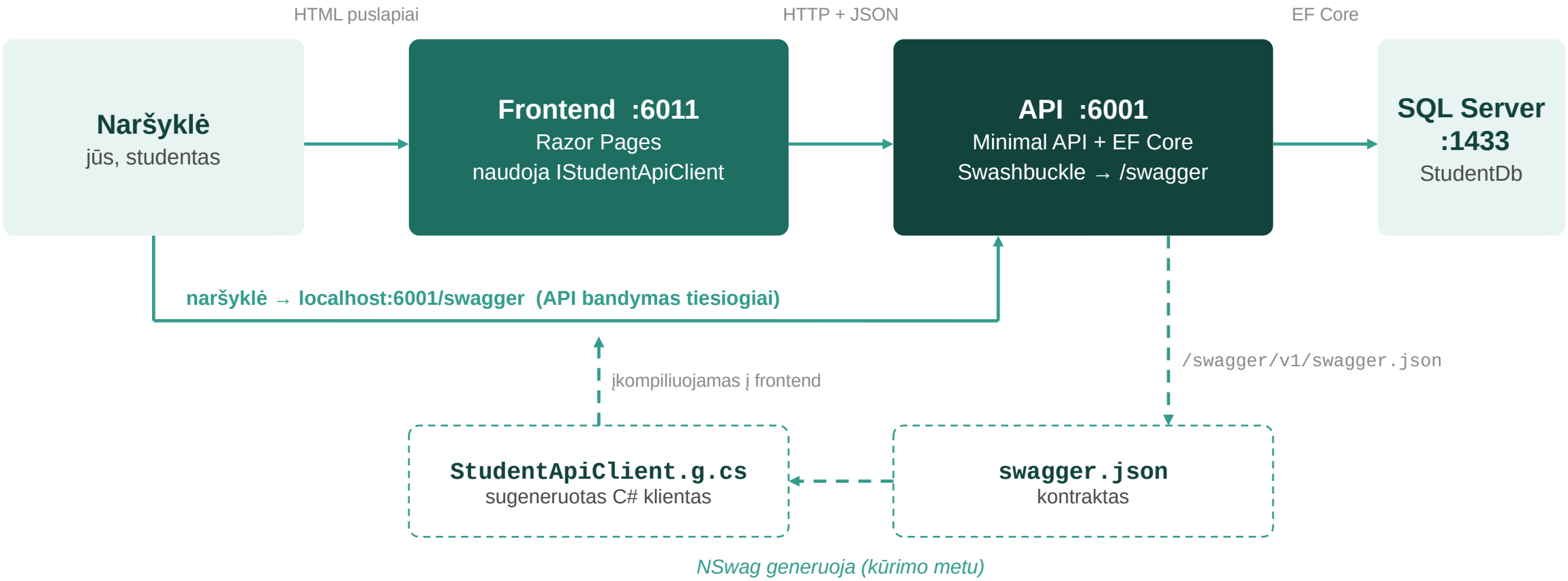
4 NSwag

Generatorius, kuris iš swagger.json sukuria tipizuotą C# (arba TypeScript) klientą. Jį naudojame frontend dalyje.

VIETA ARCHITEKTŪROJE

Swagger mūsų Docker laboratorijoje

Tie patys konteineriai – du nauji keliai



Swagger įdiegimas: 1 paketas, 4 eilutės

SimpleStudentApi/Program.cs

1. **Įdiekite paketą**
   Swashbuckle.AspNetCore iš NuGet
2. **Užregistruokite generatorių**
   AddEndpointsApiExplorer + AddSwaggerGen
3. **Pateikite JSON**
   app.UseSwagger() → /swagger/v1/swagger.json
4. **Pateikite UI**
   app.UseSwaggerUI() → /swagger

```
dotnet add package Swashbuckle.AspNetCore
```

```
// SimpleStudentApi.csproj
<PackageReference Include="Swashbuckle.AspNetCore"
                  Version="10.2.3" />

// Program.cs
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(o =>
{
    o.SwaggerDoc("v1", new OpenApiInfo
        { Title = "SimpleStudentApi", Version = "v1" });
});

var app = builder.Build();

app.UseSwagger();     // /swagger/v1/swagger.json
app.UseSwaggerUI();   // /swagger
```

Metaduomenys kodą paverčia dokumentacija

Kiekvienas iškvietimas tik papildo swagger.json – programos veikimas nesikeičia

```
// handler: validate → save → 201 Created
students.MapPost("/", CreateStudent)
   .WithName("CreateStudent")
   .WithTags("Students")
   .WithSummary("Create a new student")
   .WithDescription("The id is ignored ...")
   .Produces<Student>(201)
   .ProducesValidationProblem();   // 400

students.MapGet("/{id:int}", GetStudent)
   .WithName("GetStudent")
   .Produces<Student>(200)
   .Produces(404);
```

Kodas	Kaip tai atrodo Swagger
`.WithName("CreateStudent")`	operationId → metodas CreateStudentAsync() sugeneruotame kliente
`.WithTags("Students")`	grupės antraštė „Students“
`.WithSummary / .WithDescription`	tekstas šalia ir po API metodu
`.Produces<Student>(201)`	atsakymo kodas + JSON schema
`.ProducesValidationProblem()`	400 atsakymas su laukų klaidomis
`.ExcludeFromDescription()`	paslepia API metodą (mūsų „/“ peradresavimą)

Komentarai tampa pavyzdžiais

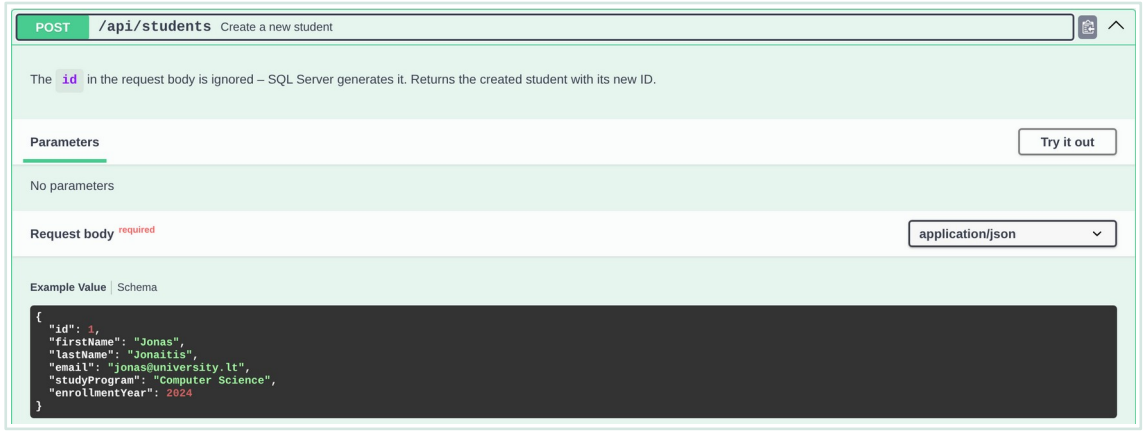
/// <summary> ir <example> modelyje → schema ir pavyzdinė reikšmė Swagger UI

```
// SimpleStudentApi.csproj
<GenerateDocumentationFile>true</GenerateDocumentationFile>

// Program.cs → inside AddSwaggerGen(o => ...)
o.IncludeXmlComments(xmlFile);

// Models/Student.cs
/// <summary>A student (table Students)</summary>
public class Student
{
    /// <example>Jonas</example>
    public string FirstName { get; set; } = "";

    /// <example>2024</example>
    public int EnrollmentYear { get; set; }
}
```

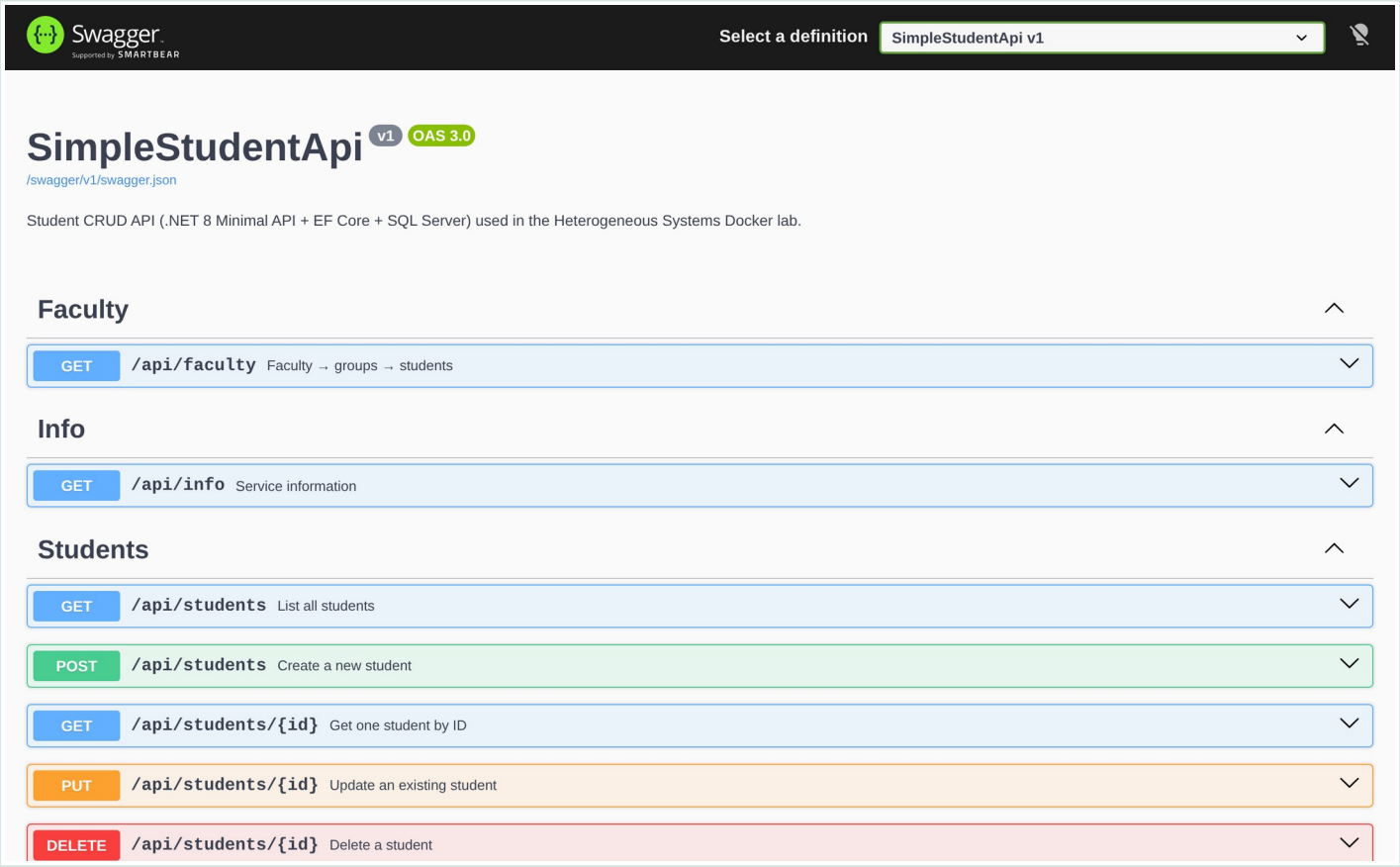


Rezultatas: aprašymas ir pavyzdinis užklausos turinys Swagger UI

! Dažna klaida: IncludeXmlComments be GenerateDocumentationFile – XML failas nesukuriamas ir API nepasileidžia.

http://localhost:6001/swagger

Visi API metodai, sugrupuoti pagal žymą, spalva pagal HTTP metodą



Žymos (tags)

grupuoja API metodus (WithTags)

Spalvos

GET mėlyna · POST žalia · PUT oranžinė · DELETE raudona

Kontrakto nuoroda

/swagger/v1/swagger.json – OpenAPI failas

Schemas

visi užklausų ir atsakymų modeliai

IŠBANDYKITE

API testavimas nerašant kliento

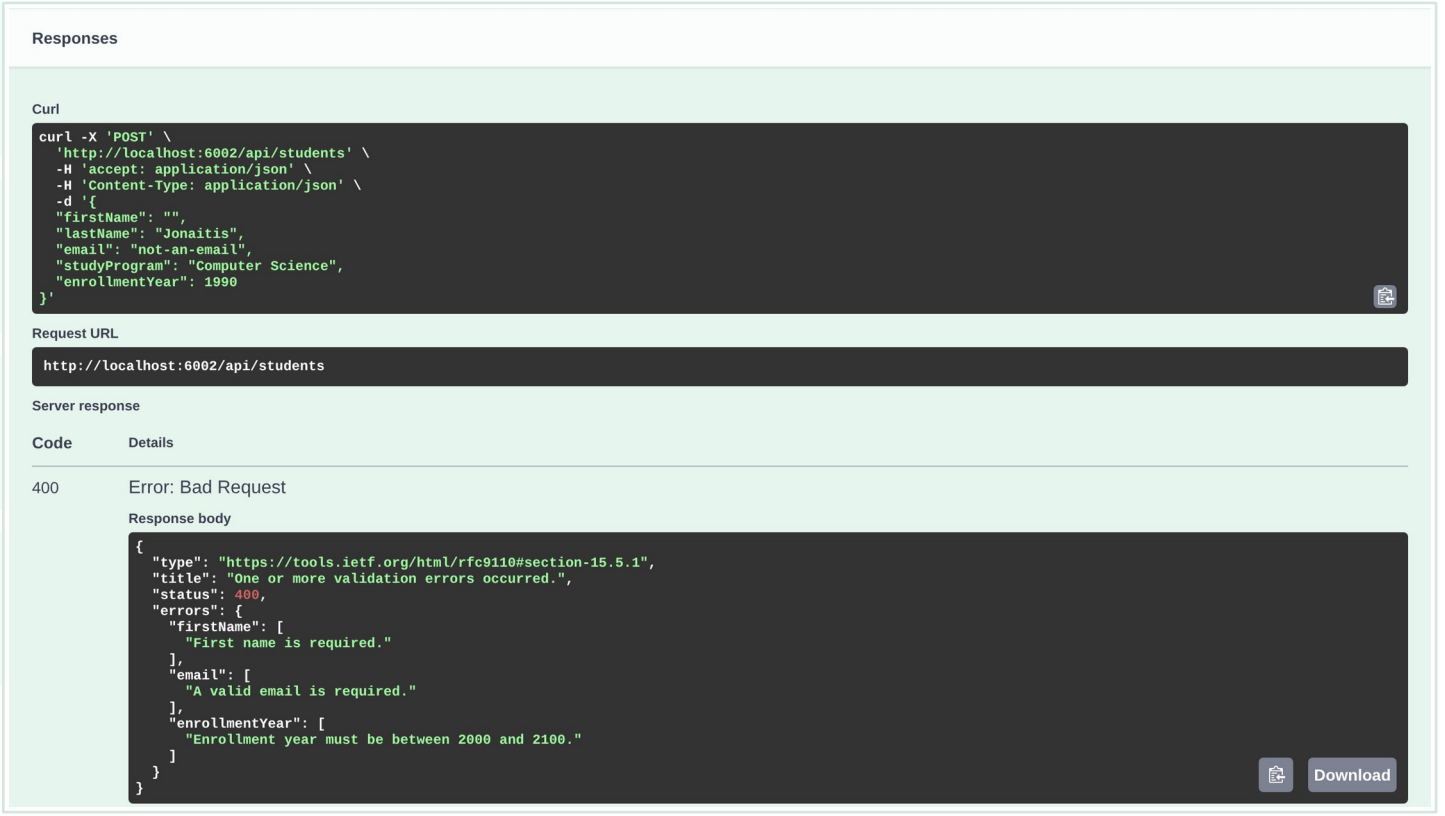
Išskleisti → Try it out → redaguoti turinį → Execute

1. **Išskleiskite POST /api/students**
   matomas aprašymas, turinio schema ir atsakymai

2. **Spauskite „Try it out“**
   pavyzdinis turinys tampa redaguojamas

3. **Siųskite neteisingus duomenis**
   tuščias vardas, 1990 m. → Execute

4. **Perskaitykite rezultatą**
   400, laukų klaidos, antraštės ir paruošta curl komanda



Swagger UI siunčia tikras užklausas – POST duomenys tikrai išsaugomi SQL Server.

KĄ GAUNATE

Vienas dokumentas – šeši panaudojimai

Kodėl verta skirti dešimt minučių konfigūracijai

1 Gyva dokumentacija

Generuojama iš kodo, todėl niekada nepasensta

2 Rankinis testavimas

Kiekvieną API metodą galima išbandyti naršyklėje – paprastiems atvejams Postman nereikia

3 Klaidų kontraktas

201 / 400 / 404 atsakymai aprašyti ir matomi

4 curl ir užklausos URL

Tikslią užklausą galima nukopijuoti į terminalą, skriptą ar klaidos aprašą

5 Klientų generavimas

NSwag arba OpenAPI Generator sukuria C#, TypeScript, Java… klientus

6 Sąsajos specifikacija

swagger.json pridedamas prie projekto dokumentacijos

Du adresai – dvi auditorijos

Konteineriai bendrauja pagal paslaugos pavadinimą, naršyklė – per kompiuterio prievadą

Docker tinklo viduje

- **Frontend → API: http://simplestudentapi:8080**
- Nustatoma ApiSettings__BaseUrl
- Naudoja sugeneruotas klientas (serverio pusėje)

Iš jūsų kompiuterio

- **Naršyklė → API: http://localhost:6001**
- Nustatoma ApiSettings__PublicUrl
- Naudojama Swagger nuorodai ir Swagger UI

Čia Swagger įjungtas visose aplinkose, nes konteineriai veikia Production režimu. Tikrame produkte – tik Development aplinkoje arba su autentifikacija.

Trys būdai kviesti API

Dvi laboratorinės parodo pirmuosius du būdus greta

A Rankinis HttpClient

```
GetFromJsonAsync<ApiStudentResponse
>("/api/getStudent")
```

+ paprasta, nereikia papildomų įrankių

– URL ir DTO rašomi ranka; klaidos randamos tik vykdymo metu

Laboratorinė: …with-additional-endpoint-and-page

B Sugeneruotas tipizuotas klientas (NSwag)

```
await _api.GetStudentsAsync()
```

+ metodai ir modeliai iš swagger.json; kontraktą tikrina kompiliatorius

– pakeitus API, klientą reikia pergeneruoti

Laboratorinė: …plus-mssql

C JavaScript naršyklėje

```
fetch("http://localhost:6001/api/
students")
```

+ SPA karkasai (React, Angular) – TypeScript klientas generuojamas taip pat

– API turi leisti CORS

Mūsų laboratorinėse nenaudojama

Kliento generavimas iš swagger.json

SimpleStudentFrontend/ApiClient/generate-client.sh (Windows: .ps1)

1. **Paleiskite API**
   docker compose up (arba dotnet run)

2. **Paleiskite generate-client**
   atsisiunčia swagger.json ir paleidžia NSwag per npx (reikia Node.js)

3. **Perkompiliuokite frontend**
   docker compose up --build

```
npx nswag openapi2csclient /runtime:Net80 \
  /input:swagger.json \
  /output:StudentApiClient.g.cs \
  /namespace:SimpleStudentFrontend.ApiClient \
  /className:StudentApiClient \
  /jsonLibrary:SystemTextJson \
  /generateClientInterfaces:true \
  /useBaseUrl:false
```

Rezultatas: IStudentApiClient, StudentApiClient ir modeliai Student, InfoResponse, FacultyResponse, ApiException – apie 1 000 eilučių, kurių nereikia rašyti ranka.

Užregistruoti kartą, naudoti visur

Program.cs, tada „prieš“ ir „po“ Students.cshtml.cs faile

```
// Program.cs
builder.Services.AddHttpClient<IStudentApiClient, StudentApiClient>(c =>
    c.BaseAddress = new Uri(apiBaseUrl));
```

Prieš (rankinis kodas)

```
var client = _httpClientFactory
    .CreateClient("StudentApi");

Students = await client
    .GetFromJsonAsync<List<StudentDto>>(
        "/api/students");

await client.DeleteAsync(
    $"/api/students/{id}");
```

Po (sugeneruotas klientas)

```
// IStudentApiClient _api is injected

Students = (await _api.GetStudentsAsync())
    .ToList();

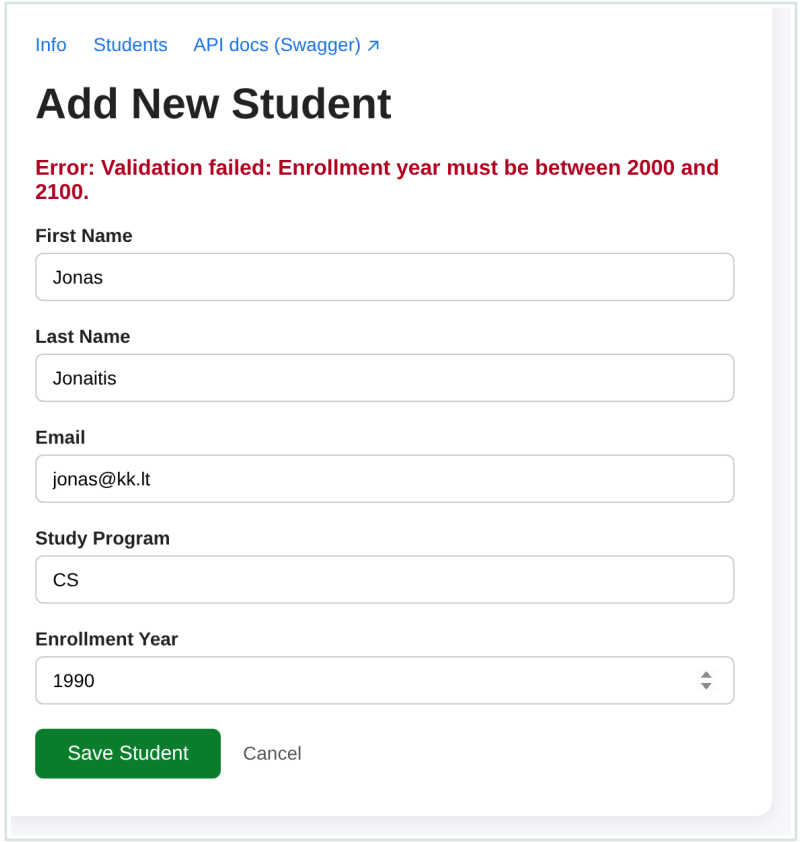
await _api.DeleteStudentAsync(id);
```

Neteisingas URL ar lauko pavadinimas → kompiliavimo klaida, o ne 404 vykdymo metu

API klaidos tampa tipizuotomis išimtimis

Swagger aprašytas 400 atsakymas C# kode gaunamas kaip ApiException<HttpValidationProblemDetails>

```
try
{
    await _api.CreateStudentAsync(Input.ToApi());
    return RedirectToPage("/Students");
}
catch (ApiException<HttpValidationProblemDetails> ex)
{
    // 400 – field errors from the API
    ErrorMessage = string.Join("; ",
        ex.Result.Errors.SelectMany(e => e.Value));
}
catch (ApiException ex) when (ex.StatusCode == 404)
{
    ErrorMessage = "Student not found";
}
```



Supaprastinta – žr. ApiClient/ApiErrors.cs

Kai API pasikeičia

Kontraktas parodo nesuderinamus pakeitimus anksčiau, nei juos pamato vartotojai

1

Pakeiskite API

pvz., email → emailAddress

→

2

Paleiskite API

swagger.json atsinaujina

→

3

Pergeneruokite klientą

`generate-client.sh`

→

4

Kompiliuokite frontend

kompiliatorius parodo visas sugadintas vietas

→

5

Ištaisykite ir diekite

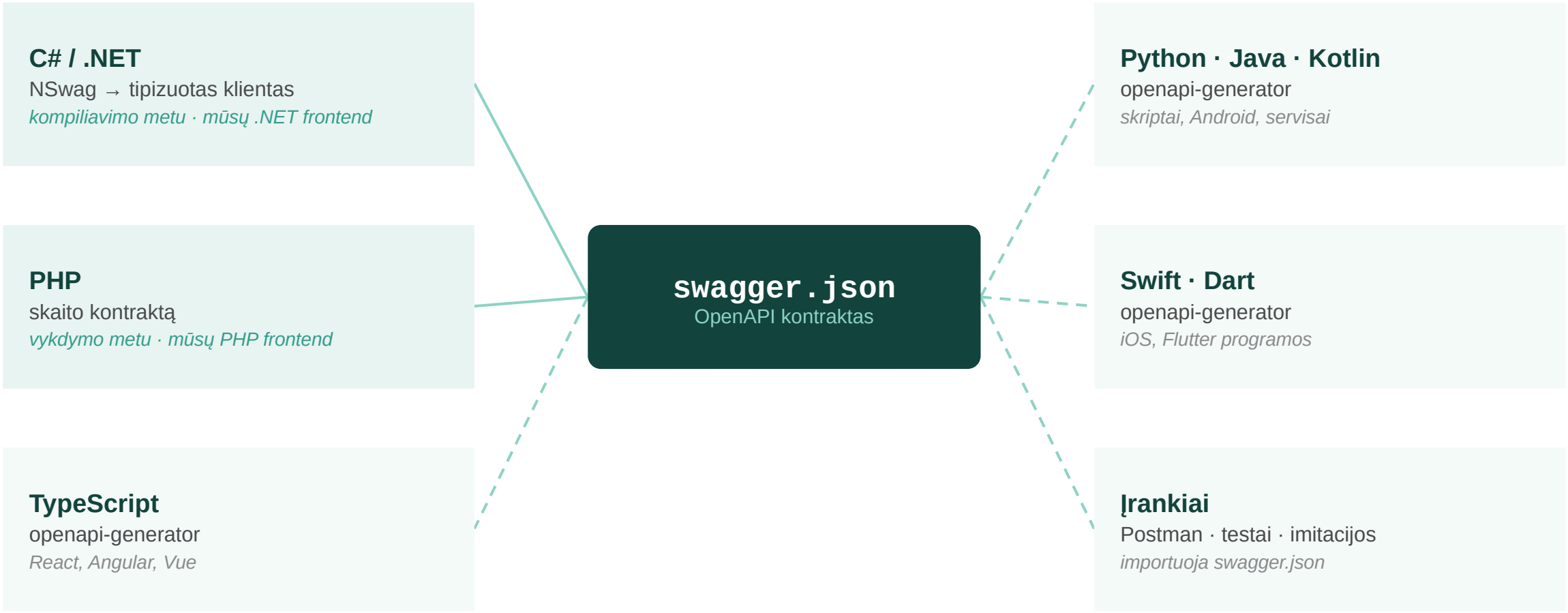
`docker compose up --build`

Be sugeneruoto kliento 4 žingsnis būtų klaida, kurią rastų vartotojas.

VIENAS KONTRAKTAS, DAUG KLIENTŲ

swagger.json tinka bet kuriai technologijai

Tas pats API, tie patys duomenys – kliento technologiją galima rinktis laisvai



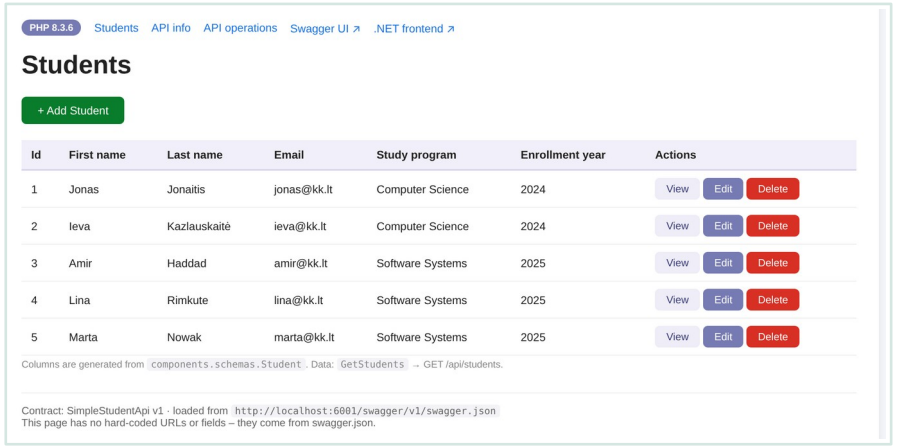
API kviečiamas pagal operacijos pavadinimą

SimpleStudentPhpFrontend/lib/ApiClient.php – paprastas PHP 8.3, be Composer paketų

```php
// public/index.php – the contract is the only configuration
$spec = OpenApi::load("$apiUrl/swagger/v1/swagger.json", ...);
$api  = new ApiClient($spec, $apiUrl);

$api->call('GetStudents');                    // GET    /api/students
$api->call('GetStudent', ['id' => 5]);        // GET    /api/students/5
$api->call('CreateStudent', [], $data);       // POST   /api/students
$api->call('UpdateStudent', ['id' => 5], $data);   // PUT
$api->call('DeleteStudent', ['id' => 5]);     // DELETE /api/students/5

// lib/ApiClient.php – inside call()
$op   = $spec->operation($operationId);       // method + path
$path = str_replace('{id}', $params['id'], $op['path']);
```

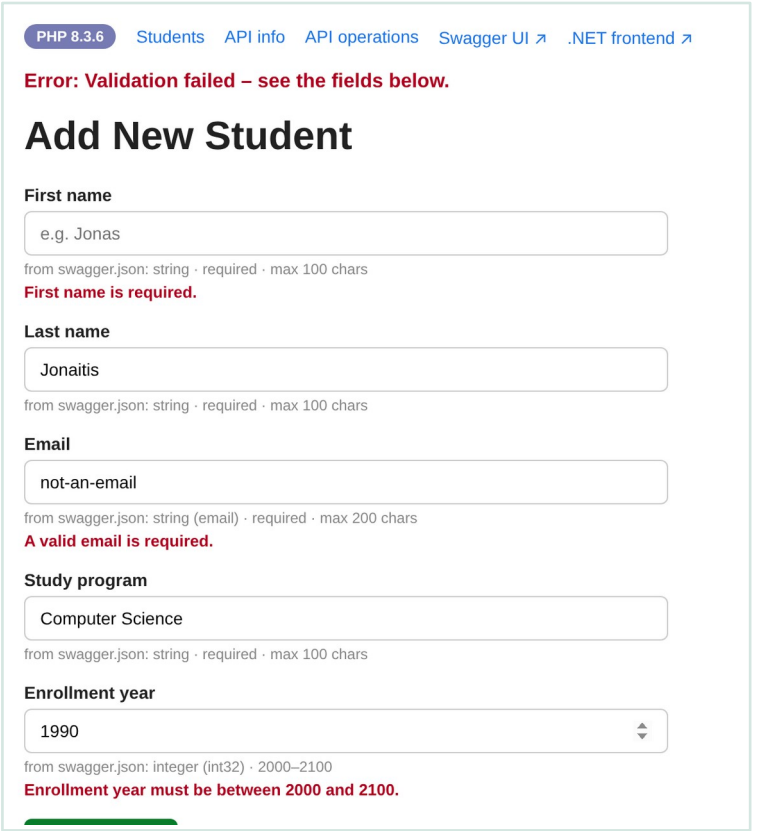


PHP frontend adresu localhost:6021 – tie patys SQL Server duomenys kaip .NET frontend

Kontraktas perduoda ir taisykles

C# atributai → swagger.json → PHP formos laukai

C# klasėje Student	swagger.json	PHP laukas
`[Required]`	`required`	`required`
`[StringLength(100)]`	`maxLength: 100`	`maxlength="100"`
`[EmailAddress]`	`format: email`	`type="email"`
`[Range(2000, 2100)]`	`minimum / maximum`	`min / max`
`[ReadOnly(true)]`	`readOnly: true`	formoje nerodomas



Tas pats 400 atsakymas kaip Swagger UI, parodytas prie kiekvieno lauko

DEMONSTRACIJA

Laukas pridedamas tik C# kode

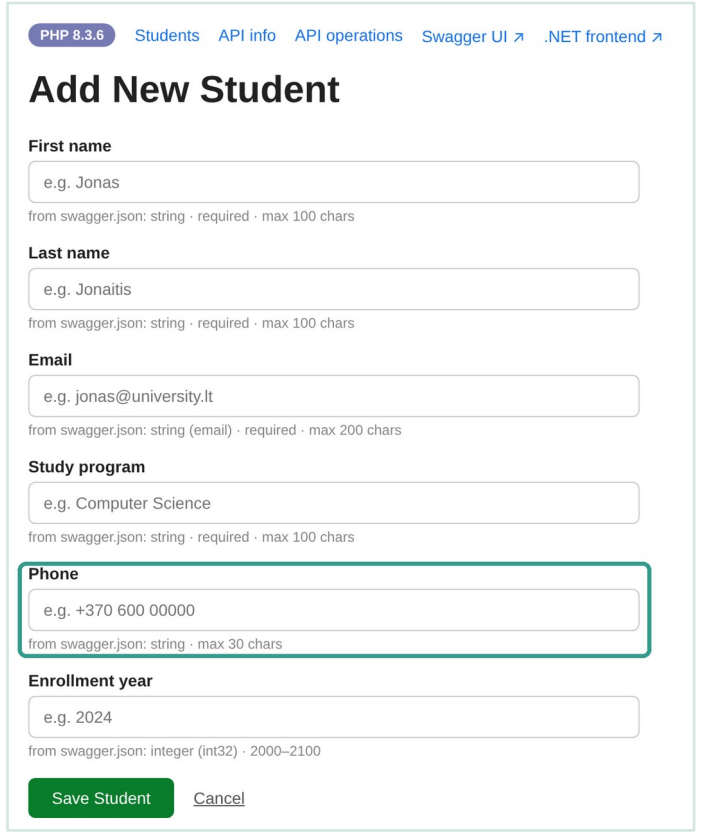
Kuris frontend pastebės pirmas?

1. **Pridėkite Phone į Student.cs**
   [StringLength(30)] public string? Phone

2. **Atkurkite duomenų bazę**
   EnsureCreated neturi migracijų – duomenys ištrinami

3. **Perkompiliuokite API**
   docker-compose up --build

4. **Atidarykite PHP frontend**
   Phone yra lentelėje ir formoje



PHP forma po 4 žingsnio – nė vienas PHP failas nepakeistas

PALYGINIMAS

Kompiliavimo ar vykdymo metu?

Du teisingi būdai panaudoti tą patį kontraktą

Sugeneruotas klientas (.NET + NSwag)

- Kodas vieną kartą generuojamas iš swagger.json
- Naujas laukas: pergeneruoti ir pakeisti puslapius
- Pervadintas laukas: kompiliavimo klaida parodo visas vietas
- **Tinka dideliems projektams, griežtam tipizavimui**

Vykdymo metu skaitantis klientas (PHP)

- Kontraktas skaitomas kiekvienos užklausos metu
- Naujas laukas: atsiranda automatiškai
- Pervadintas laukas: prisitaiko; ranka įrašyti pavadinimai sugenda vykdymo metu
- **Tinka administravimo įrankiams, prototipams**

Abu būdai remiasi tuo pačiu: tiksliu ir gerai aprašytu OpenAPI dokumentu. Geri API metaduomenys atsiperka kiekviename kliente.

PRAKTIKA

Laboratorinės užduotys (75 min.)

Dirbame su student-docker-lab-plus-mssql

1. **Paleiskite ir apžiūrėkite**
   docker compose up; atidarykite /swagger; per Try it out sukurkite 3 studentus

2. **Tyčia sugadinkite**
   išsiųskite neteisingus duomenis; palyginkite 400 atsakymą Swagger UI ir frontend formoje

3. **Pridėkite API metodą**
   GET /api/students/search?program=… su WithName, WithSummary, Produces

4. **Pergeneruokite ir naudokite**
   paleiskite generate-client; Students puslapyje pridėkite paiešką per SearchStudentsAsync

5. **Dokumentuokite**
   eksportuokite swagger.json ir įtraukite į projekto dokumentaciją kaip sąsajos specifikaciją

6. **Kita technologija**
   atidarykite PHP frontend (:6021); C# kode pridėkite Phone ir palyginkite abu frontend

OpenAPI kaip sąsajos specifikacija

Kaip tai siejasi su ankstesnių paskaitų standartais

1 ISO/IEC/IEEE 42010

Sąsajų / informacijos vaizdas: swagger.json – formalus API jungties tarp komponentų aprašas.

2 ISO/IEC 25010

Suderinamumas → sąveikumas: paskelbtas kontraktas įrodo, kad sistemos gali keistis duomenimis.

3 Europos sąveikumo sistema (EIF)

Techninis ir semantinis sąveikumas: atviras standartas (OAS) ir bendras duomenų modelis (schemos).

Dokumentacijoje: OpenAPI failas (priedas), Swagger UI ekrano kopija ir API metodų lentelė su būsenos kodais.

Swagger jūsų API paverčia kontraktu, kurį galima skaityti, testuoti ir pagal kurį galima kompiliuoti.

- Swashbuckle → swagger.json + /swagger
- Metaduomenys ir XML komentarai → naudinga dokumentacija
- NSwag → tipizuotas klientas; kontraktą tikrina kompiliatorius
- PHP (ar bet kuri kalba) → tas pats kontraktas, tie patys duomenys